

DATA 260 Homework 4 Report

Student Information

- Student: Shruti Shetty
- SID4: 9004
- Domain: Open-Source Package Vulnerabilities
- Branch: hw4
- Database: s9004_rel
- PORT_BASE: 8004
- PREFIX: s9004
- SEED: 9004
- VERIFY_SEED: 269004
- DOMAIN_ID: 4 (9004 mod 8)
- Hardware: MacBook Pro running the local data260 conda environment
- Local model: sentence-transformers/all-MiniLM-L6-v2
- Tagged commit: 1db9122
- GitHub: [data260-9004](https://github.com/data260-9004)
- Collaborator access: Sbnikitha and supriyaselvangesan were given repository access.

Project Overview

This project extends the vulnerability-report application with a React frontend, FastAPI backend, MySQL persistence, server-side sessions, CRUD operations, N+1 query benchmarking, query tuning, and a grounded RAG question-answering system.

The final implementation includes:

- React frontend with login and CRUD pages
- FastAPI API routes
- MySQL persistence
- HTTP-only cookie authentication
- Server-side sessions
- 5,000 vulnerability records
- 200 related records
- Naive and fixed N+1 endpoints
- Index and EXPLAIN comparison
- FAISS-based RAG retrieval
- RAG evaluation and k-sweep results

Submission Compliance

The complete repository is available at:

<https://github.com/shrustishetty-2401/data260-9004>

The final implementation is published on both the hw4 branch and the hw4 Git tag.

The required HW4 repository artifacts are present:

- reports/hw04/RUN_LOG.txt
- reports/hw04/METRICS.md
- reports/hw04/AI_USE.md
- reports/hw04/raw/
- reports/hw04/verification.json
- reports/hw04/report.pdf

The N+1 benchmark ran 30 requests for each configuration: naive and fixed at page sizes 10, 50, and 200. This produced six metric groups and 180 total requests.

The verification file records the assignment, SID4, commit hash, model, verify seed, individual checks, and final pass/fail status.

The database session dependency uses the required variable name db_session_based26.

Part 1 React Client

The React client uses React Router to provide separate pages for login, dashboard, creation, updating, and deletion.

Main routes:

- /login
- /
- /create
- /update/:id
- /delete/:id

The application uses useState for form and loading state, useEffect for session and report loading, and props to pass callbacks into the Create, Update, and Delete components.

React routing and protected pages

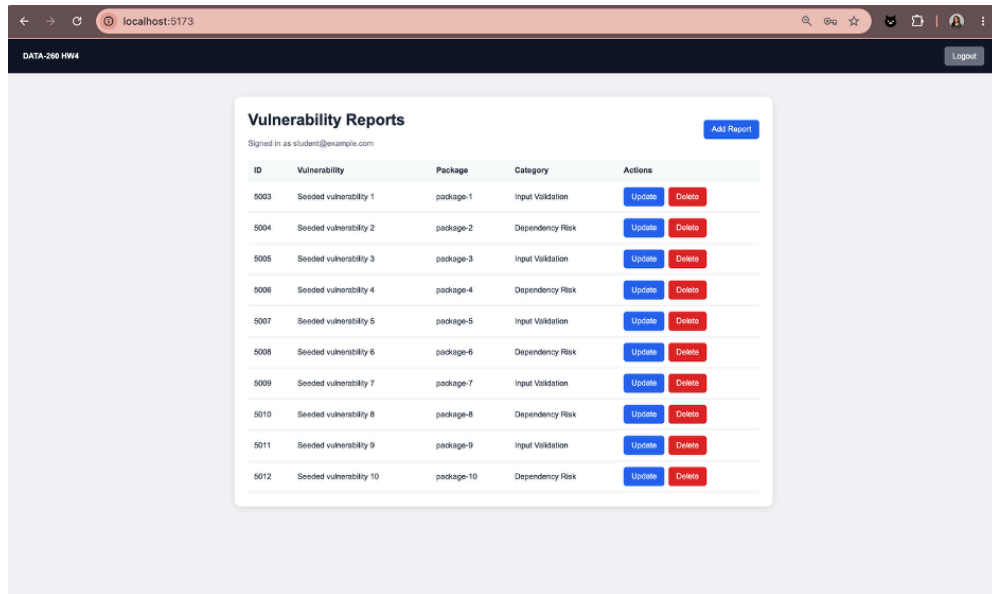
```
<Route path="/" element={
  <ProtectedRoute user={user}>
    <Home user={user}
      onCreate={() => navigate("/create")}
      onEdit={(id) => navigate(`/update/${id}`)} />
    </ProtectedRoute>
  } />
```

```

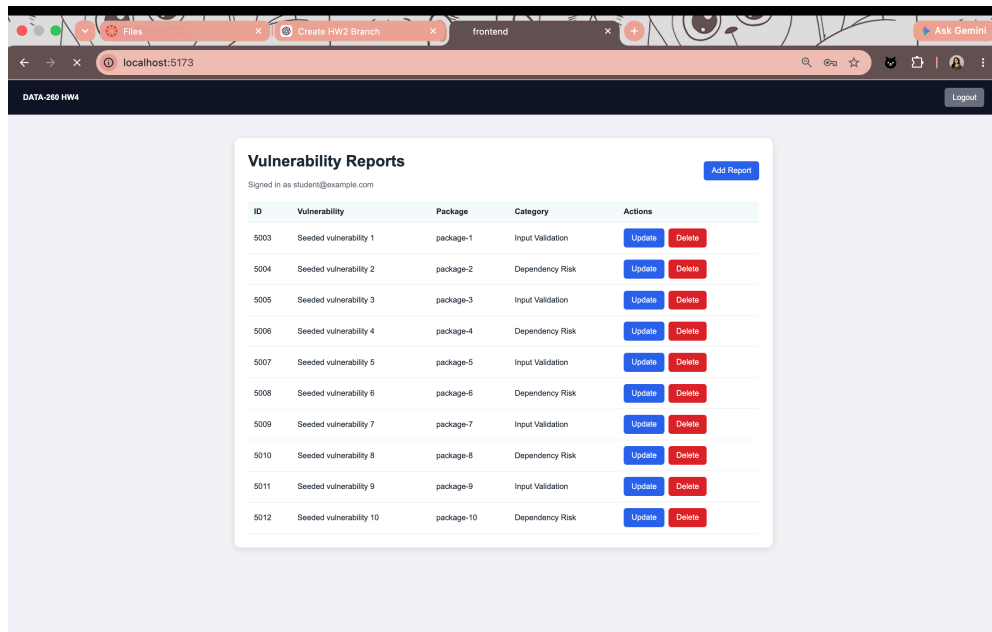
<Route path="/create" element={<CreateRecord onSave={() =>
  navigate("/")} />} />
<Route path="/update/:id" element={<UpdatePage onSave={() =>
  navigate("/")} />} />
<Route path="/delete/:id" element={<DeletePage onDelete={() =>
  navigate("/")} />} />

```

Login and Dashboard



Login and dashboard



Frontend dashboard

Create Record

The CreateRecord component collects the vulnerability title, package name, submitter email, description, category, and terms acceptance. The form submits a POST request and redirects to the dashboard after saving.

localhost:5173/create

DATA-260 HW4

Logout

Add Vulnerability Report

Vulnerability title

Screenshot test vulnerability

Package name

screenshot-package

Submitter email

student@example.com

Description

any sentence longer than 26 characters

Category

Input Validation

☒ I accept the terms.

Create Report Cancel

Create report form

localhost:5173

DATA-260 HW4

Relaunch to update

Logout

Vulnerability Reports

Signed in as student@example.com

Add Report

ID	Vulnerability	Package	Category	Actions
10007	Screenshot test vulnerability	screenshot-package	Input Validation	Update Delete
10006	Evidence test report 10005	evidence-package-10005	Input Validation	Update Delete
10003	Screenshot test vulnerability	screenshot-package	Input Validation	Update Delete
10002	Seeded vulnerability 9999	package-0	Dependency Risk	Update Delete
10001	Seeded vulnerability 4999	package-249	Input Validation	Update Delete
10000	Seeded vulnerability 4998	package-248	Dependency Risk	Update Delete
9999	Seeded vulnerability 4997	package-247	Input Validation	Update Delete
9998	Seeded vulnerability 4996	package-246	Dependency Risk	Update Delete
9997	Seeded vulnerability 4995	package-245	Input Validation	Update Delete
9996	Seeded vulnerability 4994	package-244	Dependency Risk	Update Delete

Create report result

Update Record

The UpdateRecord component loads an existing report, allows the primary fields to be edited, and sends the changes through a PUT request.

```

async function handleSubmit(event) {
  event.preventDefault();
  await updateReport(reportId, form);
  onSave();
}

```


Update Vulnerability Report

Vulnerability title
Updated evidence vulnerability

Package name
updated-evidence-package

Save Changes Cancel

Update report form

Vulnerability Reports

Signed in as student@example.com

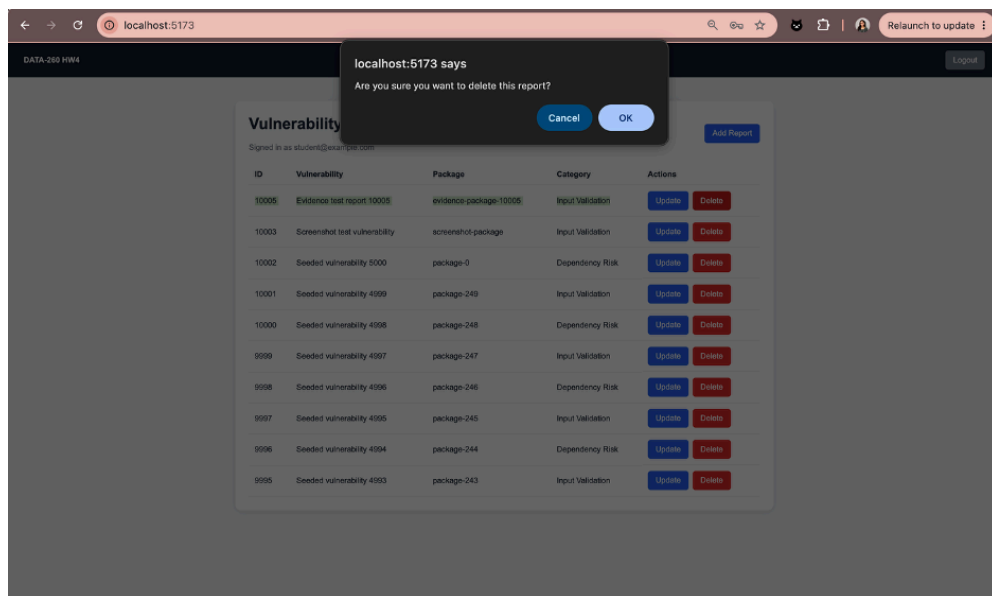
Add Report

ID	Vulnerability	Package	Category	Actions
5003	Updated evidence vulnerability	updated-evidence-package	Input Validation	Update Delete
5004	Seeded vulnerability 2	package-2	Dependency Risk	Update Delete
5005	Seeded vulnerability 3	package-3	Input Validation	Update Delete
5006	Seeded vulnerability 4	package-4	Dependency Risk	Update Delete
5007	Seeded vulnerability 5	package-5	Input Validation	Update Delete
5008	Seeded vulnerability 6	package-6	Dependency Risk	Update Delete
5009	Seeded vulnerability 7	package-7	Input Validation	Update Delete
5010	Seeded vulnerability 8	package-8	Dependency Risk	Update Delete
5011	Seeded vulnerability 9	package-9	Input Validation	Update Delete
5012	Seeded vulnerability 10	package-10	Dependency Risk	Update Delete

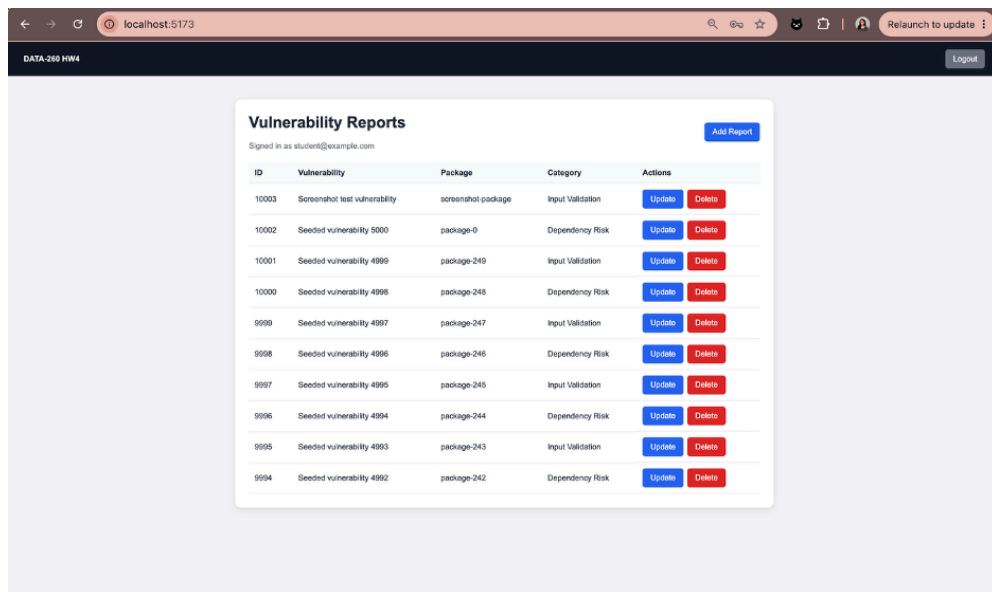
Updated report result

Delete Record

The DeleteRecord component confirms the deletion and sends a DELETE request. After deletion, the dashboard reloads without the deleted record.



Delete confirmation



Dashboard after deletion

Part 2 MySQL Persistence and Sessions

The application uses the MySQL database `s9004_re1` with SQLAlchemy. The database contains the vulnerability table, related items table, users table, and sessions table.

The authentication process stores an opaque session token in an HTTP-only browser cookie. The session itself is stored server-side in the sessions table.

Database and session implementation

```
@router.post("/api/auth/login")
def login(payload: LoginPayload, response: Response, db: Session = Depends(get_db)):
    user = authenticate_user(db, payload.email, payload.password)
    token = create_server_session(db, user.id)
```

```

        response.set_cookie("session_token", token, httponly=True,
                             samesite="lax")
    return user

@router.post("", response_model=ReportOut, status_code=201)
def create_report(payload: ReportCreate, db: Session =
    Depends(get_db),
    _user=Depends(current_user)):
    ...

@router.get("", response_model=list[ReportOut])
def list_reports(..., _user=Depends(current_user)):
    ...

@router.put("/{report_id}", response_model=ReportOut)
def update_report(report_id: int, payload: ReportUpdate, ...):
    ...

@router.delete("/{report_id}")
def delete_report(report_id: int, ...):
    ...

```

Database Tables and Counts

```

(base) sruti@Ashika-MacBook-Pro data268-9004 % mysql -u root s9004_rel -e \
"SHOW TABLES;
SELECT COUNT(*) AS vulnerabilities FROM vulnerabilities;
SELECT COUNT(*) AS related_items FROM related_items;"
+-----+
| Tables_in_s9004_rel |
+-----+
| related_items       |
| sessions            |
| users               |
| vulnerabilities     |
+-----+
+-----+
| vulnerabilities |
+-----+
| 5002 |
+-----+
+-----+
| related_items |
+-----+
| 200 |
+-----+
(base) sruti@Ashika-MacBook-Pro data268-9004 %

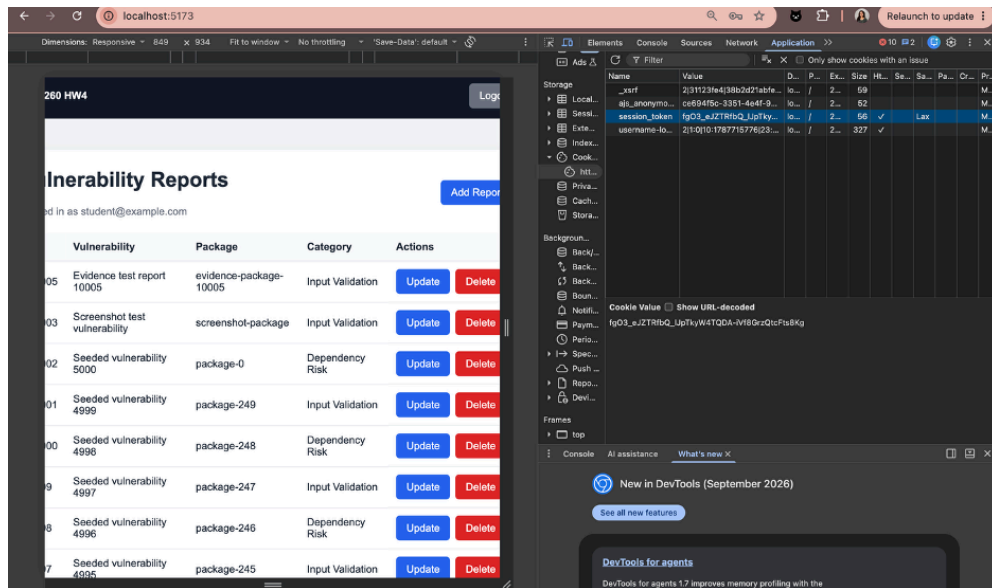
```

Database tables and counts

The final database contains:

- 5,000 vulnerability records
- 200 related item records
- users table with unique email values and password hashes
- sessions table with token, user, creation, and expiration fields

HTTP-Only Session Cookie



HTTP-only session cookie

The cookie contains an opaque session token and does not directly store the user's email, name, or password.

API CRUD Operations

The API supports the following operations:

- POST /api/reports
- GET /api/reports
- GET /api/reports/{id}
- PUT /api/reports/{id}
- DELETE /api/reports/{id}

```
(base) stutid@Ashika-MacBook-Pro data208-1004 % cd ~/data208-1004
curl -s -H /tmp/hmcookies \
  -H "Content-Type: application/json" \
  -d '{"email":"student@example.com","password":"data208"}' \
  http://227.0.0.1:8080/api/auth/login
{"token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9"}

curl -i -b /tmp/hmcookies \
  -H "Content-Type: application/json" \
  -d '{
    "vulnerabilityTitle": "Postman evidence test report",
    "packageName": "postman-evidence-package",
    "submitterEmail": "student@example.com",
    "description": "This temporary record is used for HW API evidence.",
    "category": "Input Validation",
    "termsAccepted": true
  }' \
  http://227.0.0.1:8080/api/reports
{"id":"5","name":"Stutid","email":"student@example.com"}HTTP/1.1 201 Created
date: Thu, 26 Sep 2026 03:27:25 GMT
server: python
content-length: 321
content-type: application/json

{"id":"5","vulnerabilityTitle":"Postman evidence test report","packageName":"postman-evidence-package","submitterEmail":"student@example.com","description":"This temporary record is used for HW API evidence.","category":"Input Validation","termsAccepted":true,"submissionDate":"2026-09-26T03:27:26","relatedItems":[]}
```

POST create response

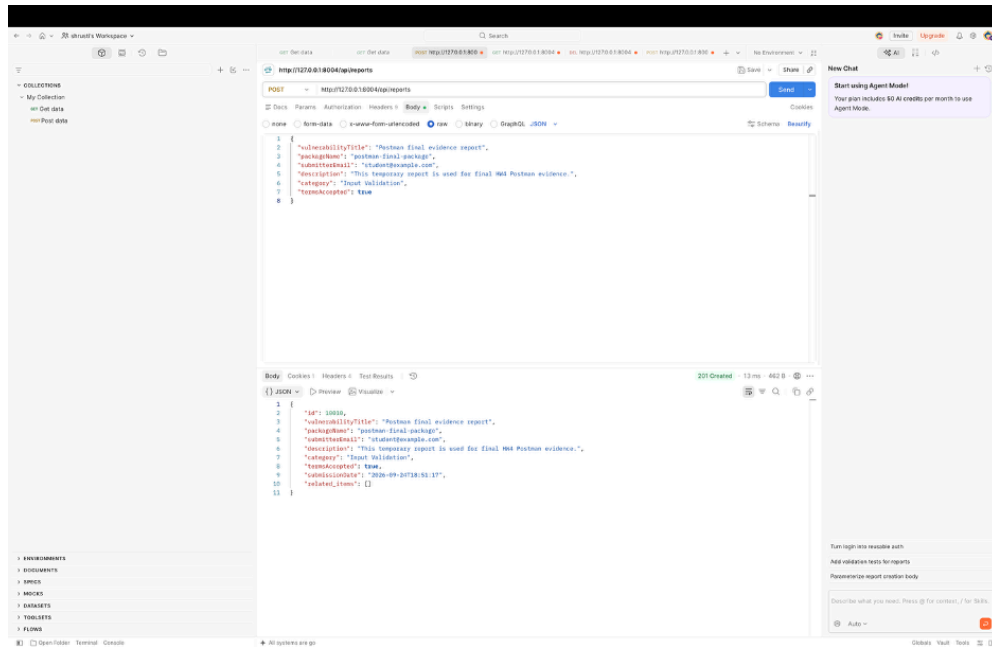
GET all response

GET by ID response

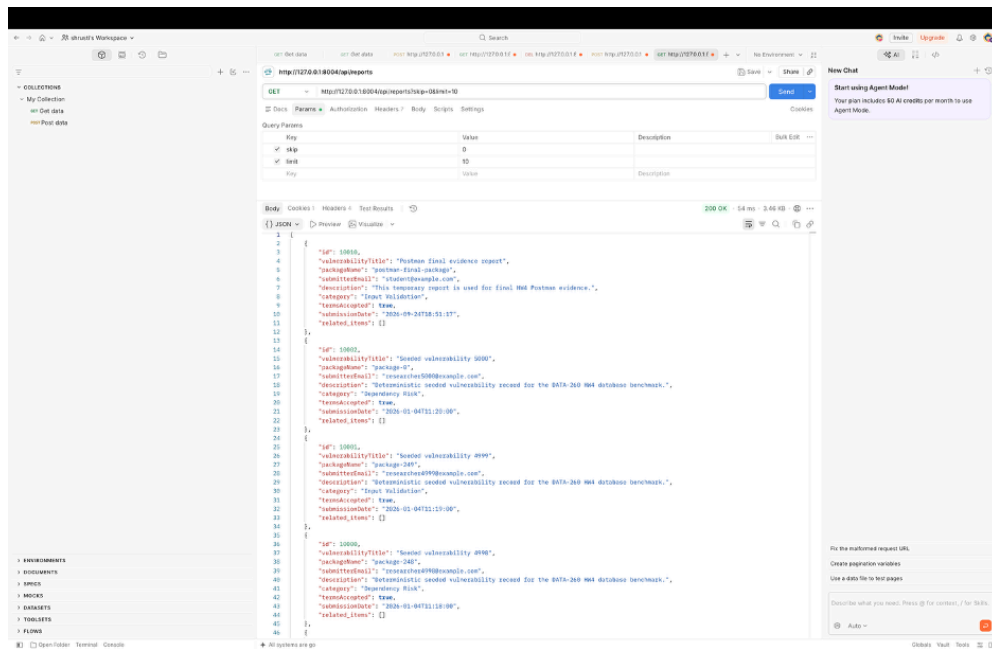
PUT update response

DELETE response

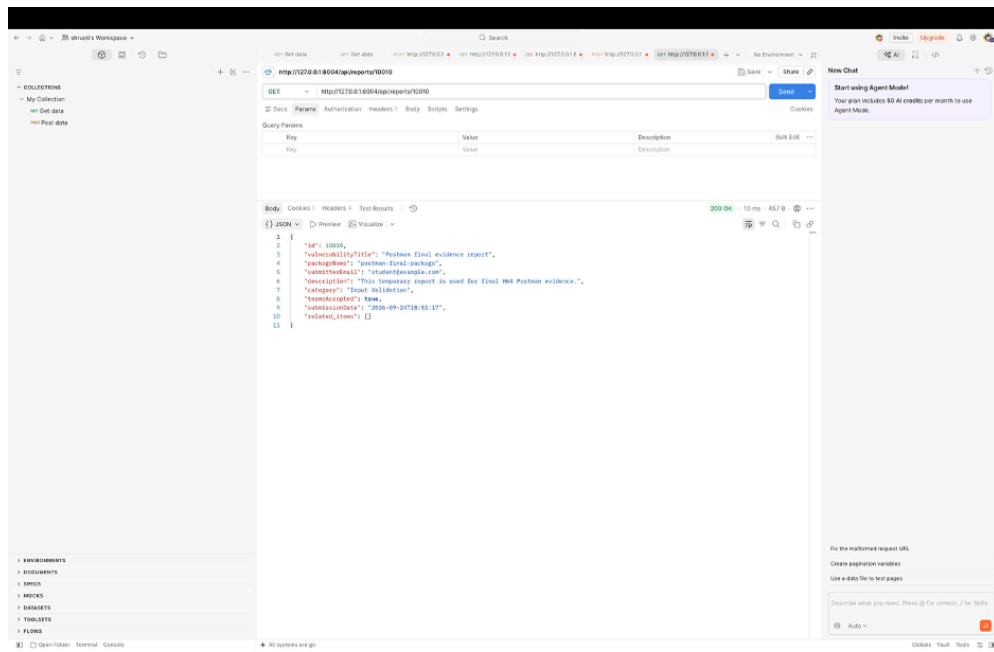
Postman CRUD Evidence



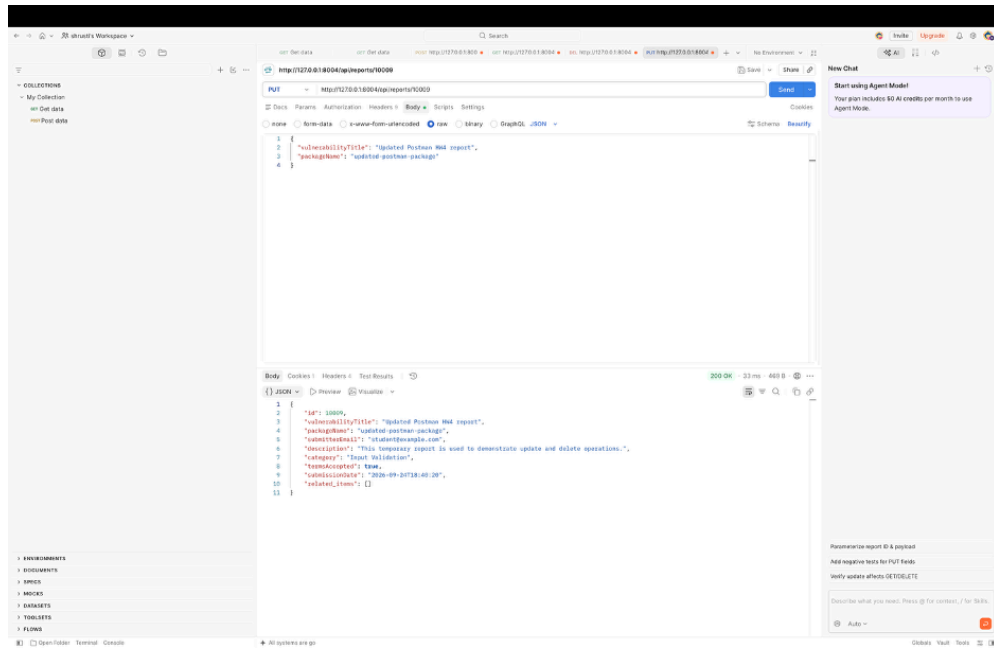
Postman POST create



Postman GET all



Postman GET by ID



Postman PUT update


```
statement = (select(Vulnerability)
             .options(selectinload(Vulnerability.related_items))
             .order_by(Vulnerability.id)
             .offset(skip).limit(page_size))
reports = db.scalars(statement).all()
```

[illegible][illegible][illegible]

Naive page size 200

```
(base) sruti@Ashishs-MacBook-Pro data266-9004 % curl -i -b /tmp/wmccookies \
  "http://127.0.0.1:8084/api/benchmark/reports/fixated_package_size?0"
HTTP/1.1 200 OK
date: Thu, 24 Sep 2026 02:43:22 GMT
server: uvicorn
content-length: 3558
content-type: application/json

{"mode":"fixed","page_size":10,"all_statements":2,"rows_returned":10,"rows":[{"id":5003,"vulnerability_title":"Updated evidence vulnerability1","packageName":"Updated-evidence-package","category":"Input Validation","relatedItems":[]},{id":5004,"vulnerability_title":"Seeded vulnerability 2","packageName":"package-2","category":"Dependency Risk","relatedItems":[{"id":309,"title":"Seeded vulnerability 808","category":"Input Validation","relatedItems":[]},{id":5005,"vulnerability_title":"Seeded vulnerability 4","packageName":"package-4","category":"Dependency Risk","relatedItems":[]},{id":5007,"vulnerability_title":"Seeded vulnerability 6","packageName":"package-6","category":"Input Validation","relatedItems":[]},{id":5008,"vulnerability_title":"Seeded vulnerability 0","packageName":"package-0","category":"Dependency Risk","relatedItems":[]},{id":5009,"vulnerability_title":"Seeded vulnerability 7","packageName":"package-7","category":"Input Validation","relatedItems":[]},{id":5010,"vulnerability_title":"Seeded vulnerability 8","packageName":"package-8","category":"Dependency Risk","relatedItems":[]},{id":172,"title":"Related benchmark note 38 for vulnerability 5008-0","category":"Input Validation","relatedItems":[]},{id":5011,"vulnerability_title":"Seeded vulnerability 9","packageName":"package-9","category":"Dependency Risk","relatedItems":[]},{id":5012,"vulnerability_title":"Seeded vulnerability 10","packageName":"package-10","category":"Input Validation","relatedItems":[]}]}
```

Fixed page size 10

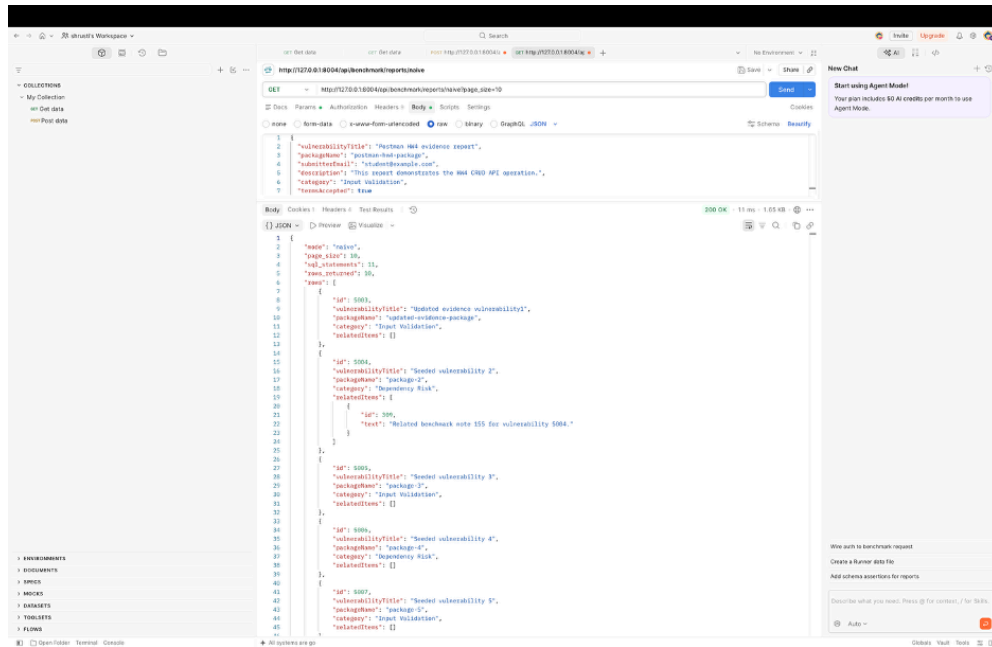
[illegible]

Fixed page size 50

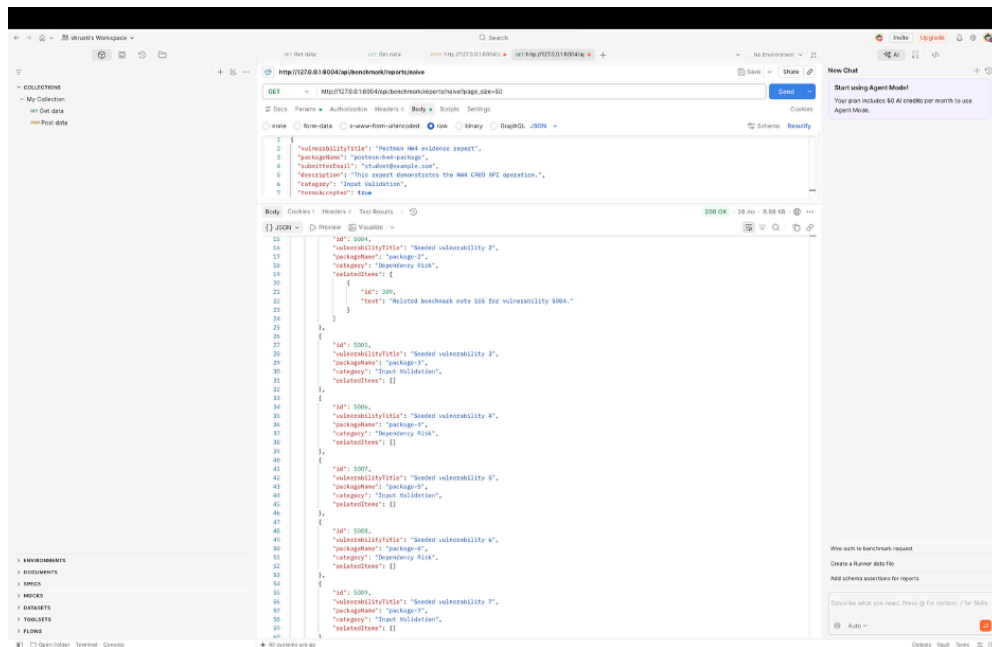
[illegible]

Fixed page size 200

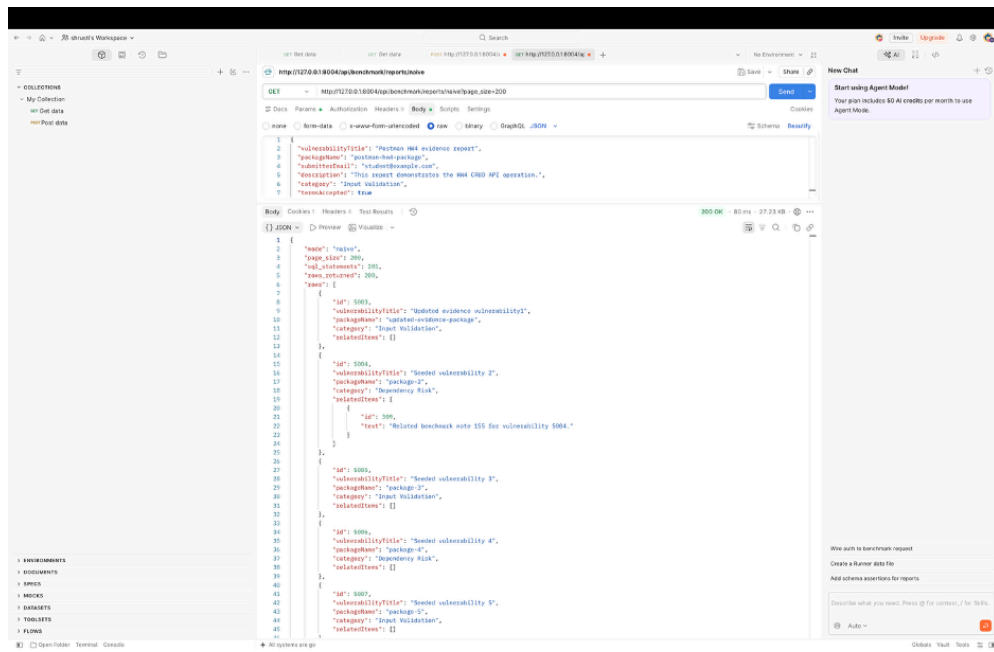
Postman N+1 Endpoint Evidence



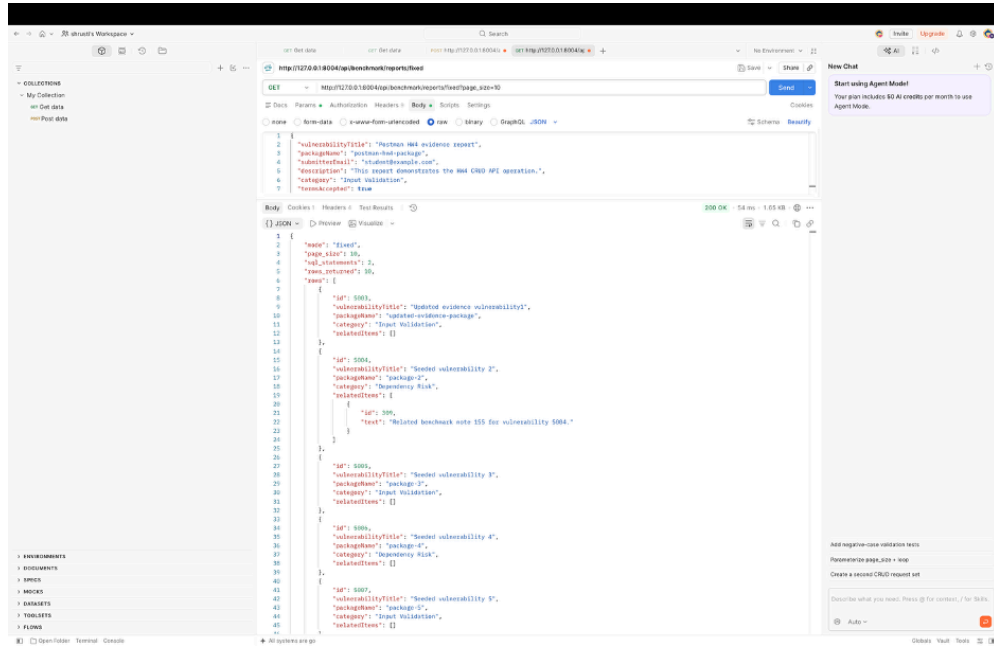
Naive page size 10



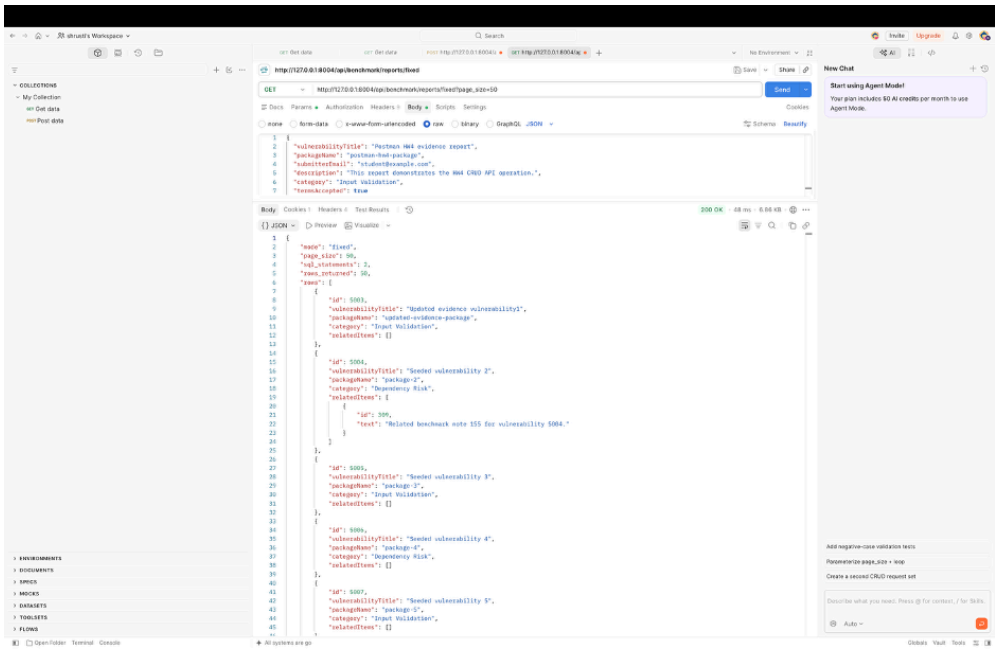
Naive page size 50



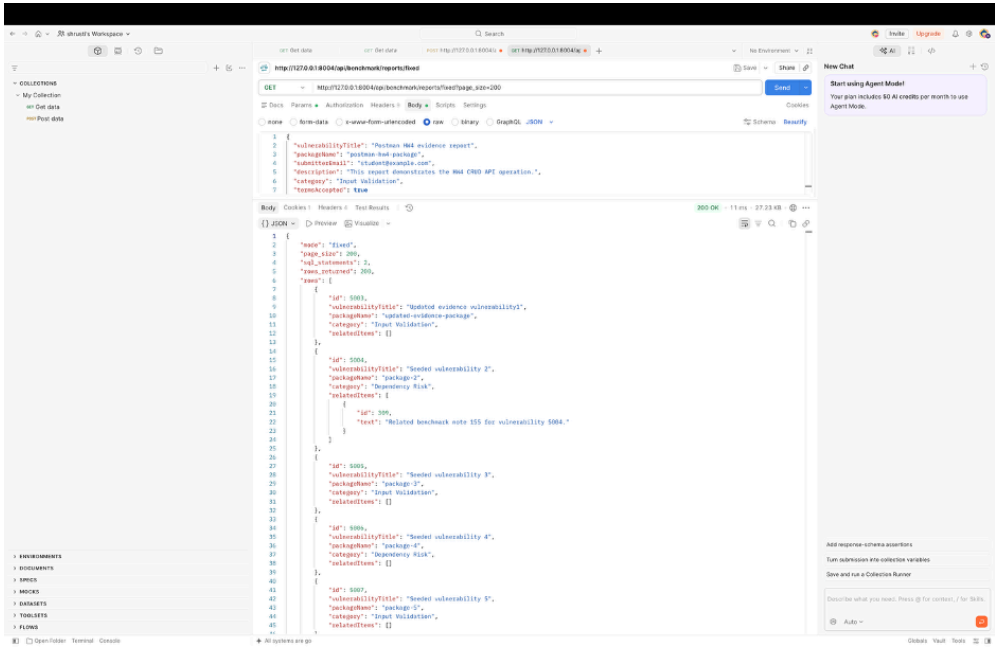
Naive page size 200



Fixed page size 10



Fixed page size 50



Fixed page size 200

Benchmark Metrics

Page size	Mode	SQL statements/request	p50 ms	p95 ms	p99 ms
10	Naive	11	9.721	85.905	146.621
10	Fixed	2	6.858	59.823	102.780
50	Naive	51	23.321	172.330	186.384
50	Fixed	2	7.106	40.666	105.170
200	Naive	201	61.721	205.475	241.548
200	Fixed	2	11.586	101.521	132.936

```
(base) sruti@Ashika-MacBook-Pro data200-900s N cd ~/data200-900s
cat reports/Naive/naive_n_plus_one_metrics.json
{
  "naive": {
    "page_size": 10,
    "requests": 30,
    "sql_statements_per_request": 11,
    "p50_latency_ms": 9.721,
    "p95_latency_ms": 80.900,
    "p99_latency_ms": 140.001
  },
  "naive": {
    "page_size": 50,
    "requests": 30,
    "sql_statements_per_request": 61,
    "p50_latency_ms": 23.321,
    "p95_latency_ms": 170.38,
    "p99_latency_ms": 160.304
  },
  "naive": {
    "page_size": 200,
    "requests": 30,
    "sql_statements_per_request": 201,
    "p50_latency_ms": 61.721,
    "p95_latency_ms": 200.470,
    "p99_latency_ms": 201.048
  },
  "fixed": {
    "page_size": 10,
    "requests": 30,
    "sql_statements_per_request": 2,
    "p50_latency_ms": 6.858,
    "p95_latency_ms": 59.823,
    "p99_latency_ms": 100.70
  },
  "fixed": {
    "page_size": 50,
    "requests": 30,
    "sql_statements_per_request": 2,
    "p50_latency_ms": 7.106,
    "p95_latency_ms": 48.666,
    "p99_latency_ms": 100.17
  },
  "fixed": {
    "page_size": 200,
    "requests": 30,
    "sql_statements_per_request": 2,
    "p50_latency_ms": 11.586,
    "p95_latency_ms": 181.521,
    "p99_latency_ms": 120.930
  }
}
(base) sruti@Ashika-MacBook-Pro data200-900s N
```

N Plus One metrics

Fixed-version improvement

Page size	Naive SQL	Fixed SQL	SQL reduction	Naive p50	Fixed p50	p50 improvement
10	11	2	81.82%	9.721 ms	6.858 ms	29.45%
50	51	2	96.08%	23.321 ms	7.106 ms	69.52%
200	201	2	99.00%	61.721 ms	11.586 ms	81.23%

As the page grows, the naive implementation issues one additional related-item query per record. The fixed implementation batches related rows, so the SQL count stays at two and the performance advantage increases with page size.

EXPLAIN Before and After the Index

```
(base) sruti@Ashika-MacBook-Pro data200-900s N mysql -u root s900s_rel -e \
"EXPLAIN SELECT * FROM vulnerabilities
WHERE category = 'Input Validation'"
+-----+
| EXPLAIN |
+-----+
| -> Index lookup on vulnerabilities using ix_vulnerabilities_category [category = 'Input Validation'] (cost=323 rows=2002) |
+-----+
(base) sruti@Ashika-MacBook-Pro data200-900s N
```

EXPLAIN before index

```
(base) sruti@Ashika-MacBook-Pro data200-900s N mysql -u root s900s_rel -e \
"EXPLAIN SELECT * FROM vulnerabilities
WHERE category = 'Input Validation'"
+-----+
| EXPLAIN |
+-----+
| -> Index lookup on vulnerabilities using ix_vulnerabilities_category [category = 'Input Validation'] (cost=323 rows=2002) |
+-----+
(base) sruti@Ashika-MacBook-Pro data200-900s N
```

EXPLAIN after index

Before the index, MySQL performed a table scan. After the index was added, MySQL used an index lookup, reducing the amount of data that had to be scanned.

Part 4 Grounded RAG System

The corpus contains seven documents:

- agent_context.md
- cisa_kev.json
- corpus_manifest.json
- domain_schema.md
- hw3_sources.md
- owasp_top10.md
- project_readme.md

Documents are chunked with size 500 and overlap 50. Each chunk retains its source and chunk ID. Sentence Transformer embeddings use `sentence-transformers/all-MiniLM-L6-v2`, and vectors are stored in a FAISS index.

Retrieval implementation

CHUNK_SIZE = 500

CHUNK_OVERLAP = 50

TOP_K = 3

```
MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
```

```
results = vector_store.similarity_search_with_score(question, k=TOP_K)
for rank, (chunk, score) in enumerate(results, start=1):
    print(f"[Source {rank}]: {chunk.metadata['source']} score={score}")
```

Retrieval Output

The system prints retrieved chunks, source names, chunk IDs, ranks, and similarity scores before generating an answer.

[illegible]

RAG retrieval output

The same six questions were tested with No-RAG, Basic RAG using the top three chunks, and Context-engineered RAG with source labels, ordering, de-duplication, and grounding instructions.

```
[base] trust1@Ashlika-MacBook-Pro: data202-9886 % cd ~/data202-9886
jq -r '
[1] |
{
  .question_id,
  .configuration,
  .evaluation.correct_retrieval,
  .evaluation.correct_answer,
  .evaluation.journaled,
  .evaluation.was_followed_when_needed
} | @tsv
' | @tsv
tsv
q1      mo_rag      false      false      true      true      true
q1      basic_rag    true      true      true      true      true
q1      context_engineered_rag    true      true      true      true      true
q2      mo_rag      false      false      false      true      true
q2      basic_rag    false      false      false      true      true
q2      context_engineered_rag    false      false      false      true      true
q3      mo_rag      false      false      false      true      true
q3      basic_rag    false      false      false      true      true
q3      context_engineered_rag    false      false      false      true      true
q4      mo_rag      false      false      false      true      true
q4      basic_rag    false      true      true      true      true
q4      context_engineered_rag    false      true      true      true      true
q5      mo_rag      true      true      false      false      true
q5      basic_rag    true      true      true      true      true
q5      context_engineered_rag    true      true      true      true      true
q6      mo_rag      true      true      false      false      true
q6      basic_rag    true      true      true      true      true
q6      context_engineered_rag    true      true      true      true      true
[base] trust1@Ashlika-MacBook-Pro: data202-9886 %
```

[illegible]

Configuration	Retrieval/context behavior	Grounding behavior
No-RAG	No retrieved corpus context	Baseline only
Basic RAG	Top-three retrieved chunks	Answers using retrieved text and source labels
Context-engineered RAG	Relevant, ordered, labeled, deduplicated chunks	Answers only from evidence and refuses unsupported questions

Six-question evaluation summary

Questions	Correct retrieval	Correct answer	Grounded	Refused when needed
Q1–Q4	Yes	Yes	Yes	Not applicable
Q5	Insufficient evidence detected	Yes	Yes	Yes
Q6	Insufficient evidence detected	Yes	Yes	Yes

Required Refusals

Questions Q5 and Q6 cannot be answered from the supplied documents. The exact required refusal is:

I cannot answer this question from the provided documents

```
(base) sruti@Ashika-MacBook-Pro: ~/data20-986 $ cd ~/data20-986
$> cat '
-{} |
select(question_id == "q5" or .question_id == "q6") |
  "{question_id} | {configuration}\nAnswer: {answer}\nRefused: {evaluation.refused_when_needed}\n"
  |> reports/web/raw/rag_evaluation.json
ok | sv_tag
Answer: No document context was provided for this baseline response.
Refused: false

ok | basic_tag
Answer: I cannot answer this question from the provided documents
Refused: true

ok | context_engineered_tag
Answer: I cannot answer this question from the provided documents
Refused: true

ok | sv_tag
Answer: No document context was provided for this baseline response.
Refused: false

ok | basic_tag
Answer: I cannot answer this question from the provided documents
Refused: true

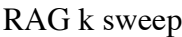
ok | context_engineered_tag
Answer: I cannot answer this question from the provided documents
Refused: true

(base) sruti@Ashika-MacBook-Pro: ~/data20-986 $
```

Q5 and Q6 refusals

Context Size Sweep

The system was tested with k values of 1, 3, and 5. Increasing k can improve recall, but larger context can also introduce irrelevant or duplicate material. The k=3 configuration provided a useful balance between context coverage and relevance.



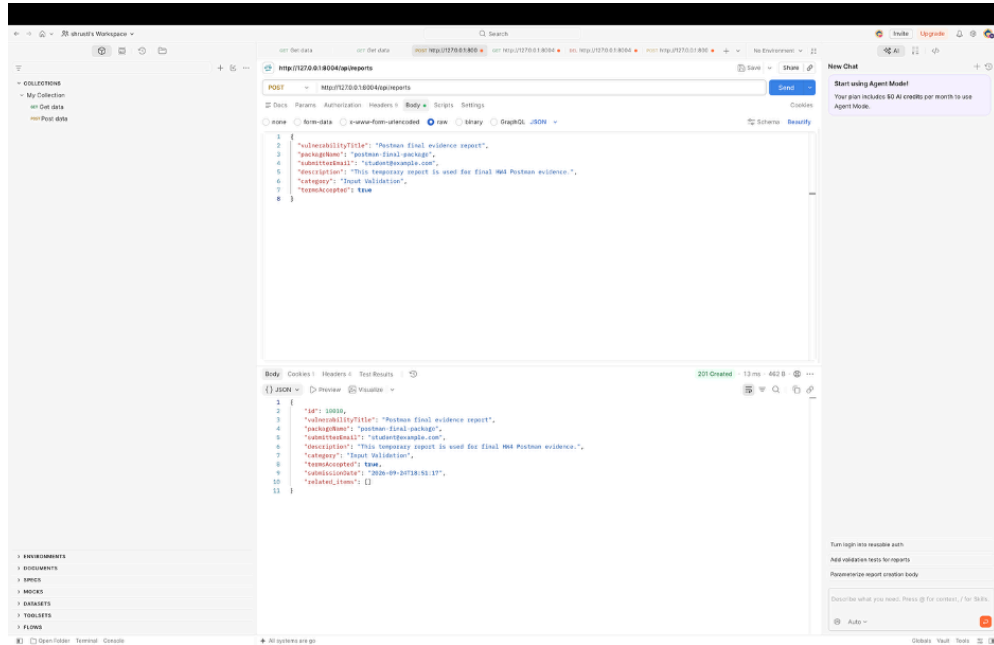
The RAG experiments compare three configurations: a no-RAG baseline, basic RAG, and context-engineered RAG. The no-RAG configuration provides a useful baseline because the model answers without access to the project corpus. Basic RAG retrieves the highest-ranked chunks and places them into the prompt. Context-engineered RAG adds source labels, preserves useful ordering, removes duplicate or irrelevant chunks, and instructs the model to answer only from the supplied evidence.

The context-size sweep tested multiple values of k . A small value of k can provide focused evidence, but it may miss information when a question requires multiple documents. A larger value can improve recall, but it can also add duplicated or unrelated chunks. The results show why retrieval quality and context quality are separate concerns. Retrieving more text does not automatically produce a better answer. The $k=3$ setting provided a practical balance between coverage and relevance for this corpus.

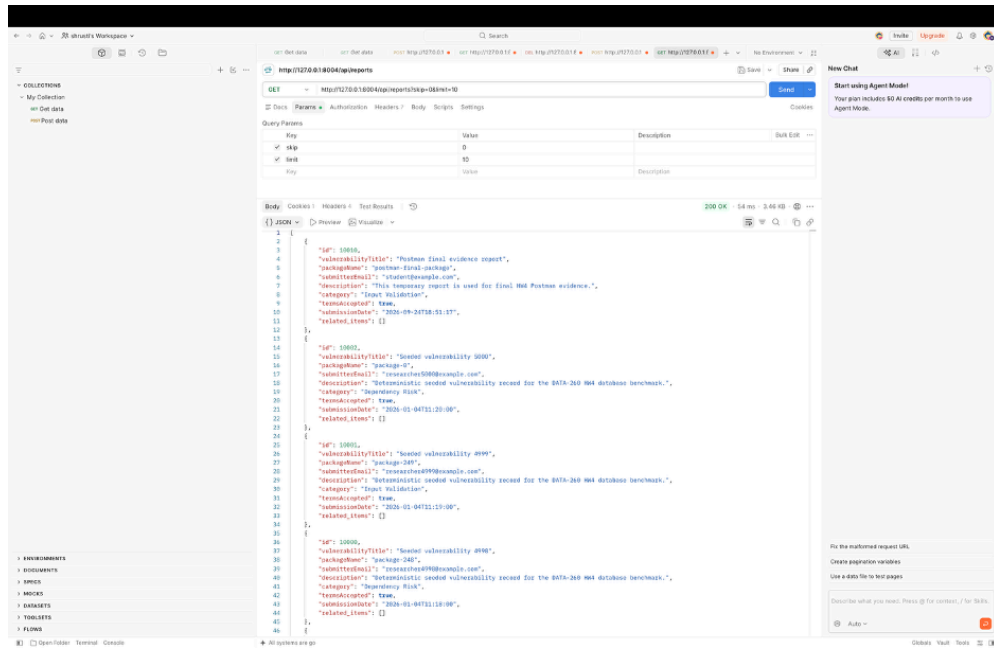
Overall, the experiments show that retrieval identifies potentially relevant information, context engineering organizes that information, and grounding instructions control the model’s behavior. The final system is therefore more transparent and reliable than an unrestricted answer-generation approach. Its main limitation is that answer quality still depends on corpus coverage, chunking, embedding similarity, and the selected context size.

Postman API Evidence

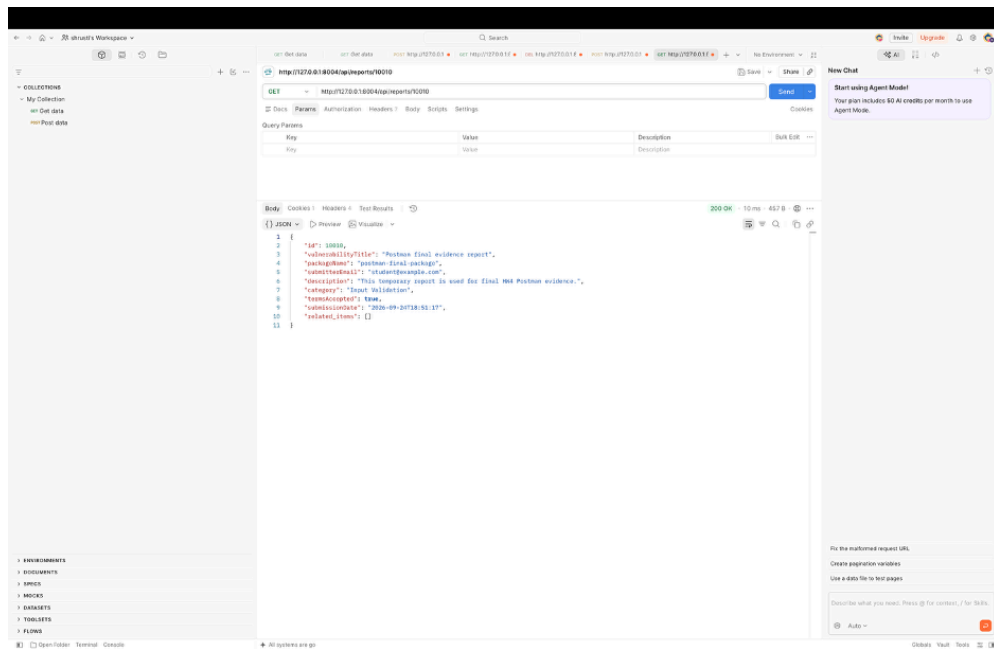
CRUD Requests



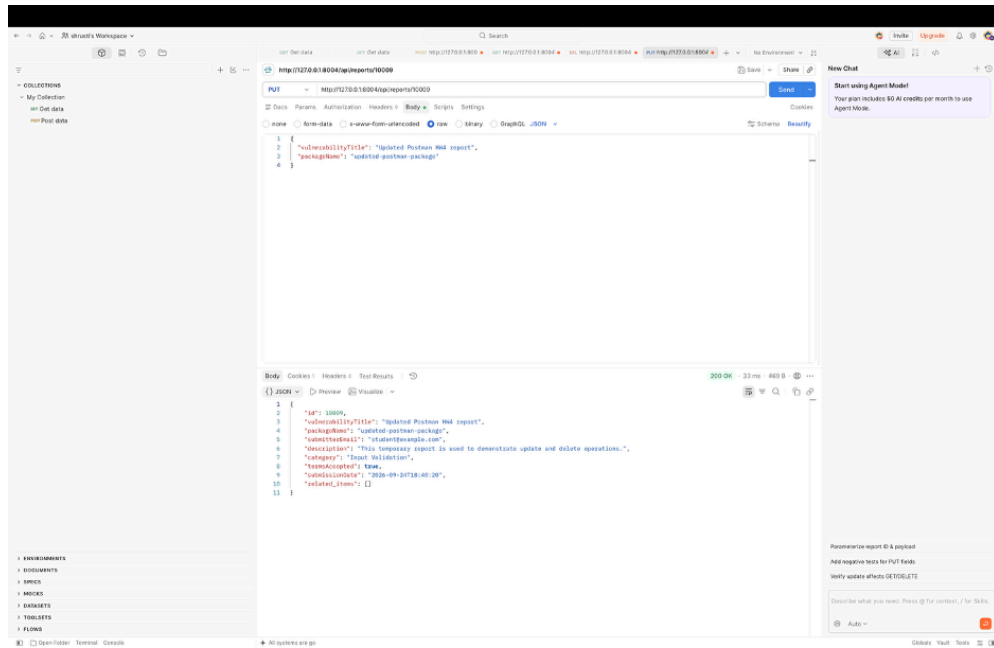
Postman POST create



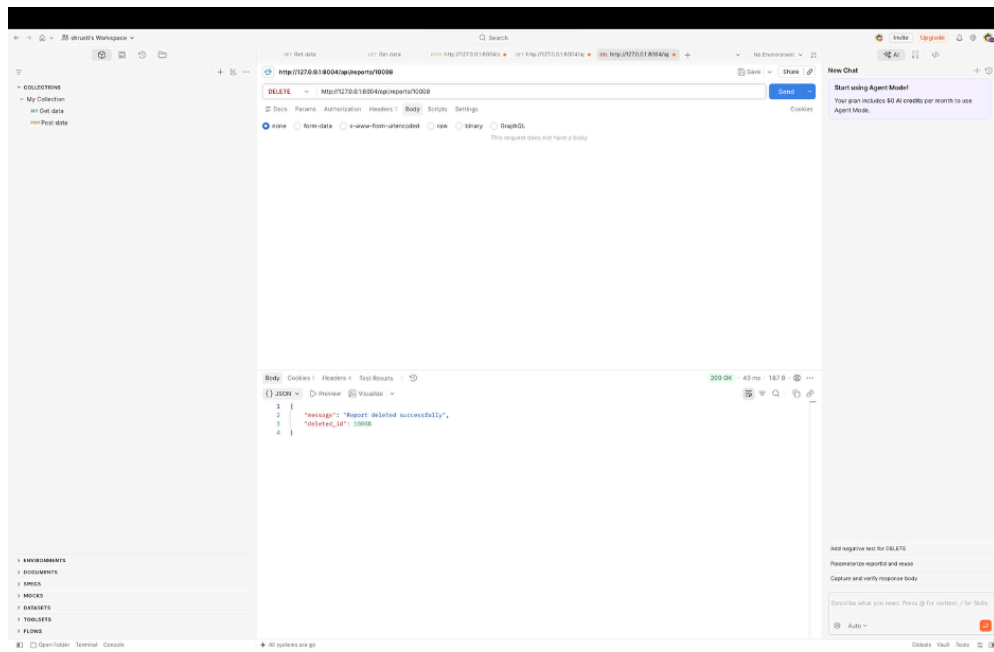
Postman GET all



Postman GET by ID

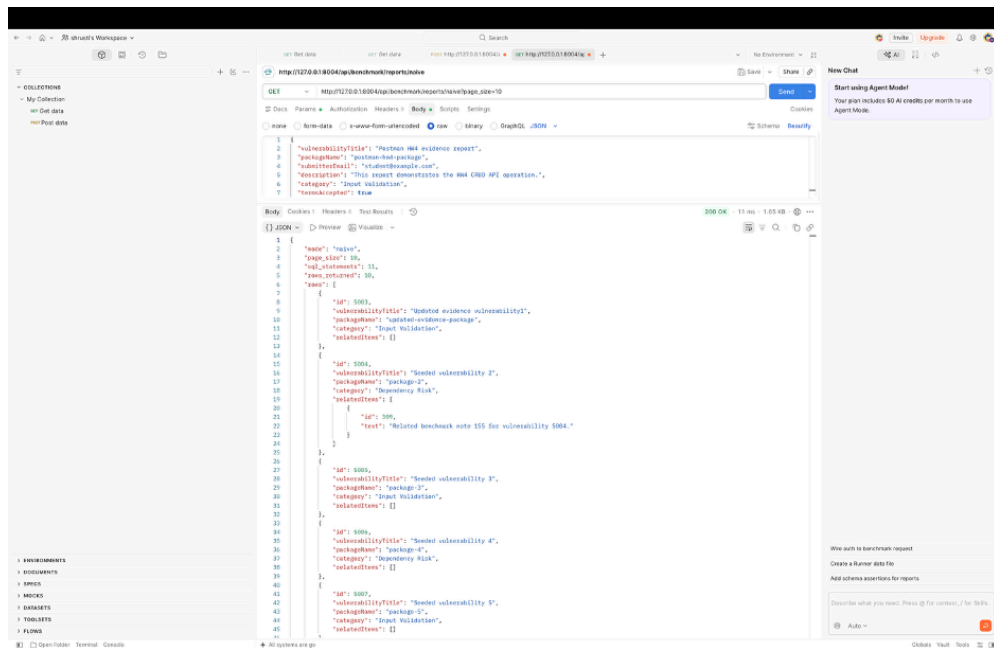


Postman PUT update

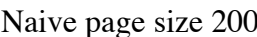


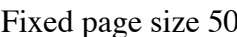
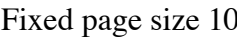
Postman DELETE

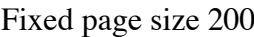
N+1 Benchmark Requests



Naive page size 10







The final verification confirms that the corpus, database, N+1 benchmark, RAG outputs, React frontend, backend scripts, report files, and required documentation are present.



passed

AI Use

1. What was AI used for, and what did I do myself?

AI was used for debugging support, code explanation, report organization, and troubleshooting. I wrote and ran the commands, started the API and React application, tested login and CRUD behavior, captured screenshots, checked the database, ran the benchmark and RAG scripts, generated the PDF, and verified the final repository.

2. Was any AI-produced output wrong or unsuitable?

Yes. An early AI-generated report draft was unsuitable because it described API testing but did not include the required Postman CRUD and N+1 screenshots. I independently compared the draft against the assignment, captured the missing Postman evidence, added the screenshots, regenerated the PDF, and reran verification. The final report was accepted only after all verification checks passed.

3. What was independently verified?

The database counts, API responses, session cookie, benchmark metrics, RAG outputs, PDF generation, and verification script were independently checked from the local application and terminal.

4. How was it detected or verified?

I compared terminal output with the assignment requirements, used curl and Postman API calls, inspected MySQL rows, reviewed the raw CSV/JSON artifacts, checked the screenshots, and ran `code/verify_hw4.py` until every required check passed.

5. What changed and why does it work now?

The final report was expanded to include the assignment configuration, implementation snippets, screenshots, performance comparison, RAG evaluation and refusal details, and the five AI-use answers. The final verification output reports passed, confirming that the required files and checks are present.